UNIVERSITÉ DE YAOUNDÉ I

École Nationale Supérieure Polytechnique de Yaoundé
Département de Génie Informatique

# Rapport de tests

## Éditeur collaboratif de documents

Tests fonctionnels, cohérence et mesures de charge locales

Étudiant : OSSOMBE Pierre René Raoul
Formation : Génie informatique — 5e année
Encadrement : Pr Jean Marc Menaud
Date de campagne : 26 octobre 2026

# Résumé exécutif

La campagne confirme le bon fonctionnement des principaux contrôles métier sur l'environnement local : les trois tests d'intégration passent, la compilation de l'API et de l'interface réussit, les 300 connexions WebSocket testées reçoivent leur état initial, et un PDF généré a été téléchargé et reconnu comme fichier PDF valide.

Les mesures HTTP portent sur des lectures locales et ne représentent pas le parcours complet de 15 000 visiteurs. Les 300 connexions WebSocket provenaient d'un seul compte de test. La capacité exigée de 15 000 visiteurs, dont 12 000 dans la première minute, avec 300 rédacteurs actifs, reste donc à démontrer sur l'infrastructure cible. Ce rapport ne revendique pas cette capacité.

# Table des matières

**1 Objet et périmètre** **1**

1.1 Précautions de campagne . . . . . 1

**2 Méthode et critères** **2**

**3 Résultats** **3**

3.1 Compilation et tests d'intégration . . . . . 3

3.2 Lectures HTTP concurrentes . . . . . 3

3.3 Connexions WebSocket . . . . . 3

3.4 Réservation et fichier PDF . . . . . 4

**4 Analyse et limites** **5**

**5 Conclusion** **6**

**A Reproduction des vérifications** **7**

**B Pièces justificatives** **8**

**C Captures visuelles des résultats** **9**

**D Justificatif des coûts AWS** **11**

# 1 Objet et périmètre

Ce rapport consigne les vérifications réalisées sur l'application de rédaction collaborative. La campagne couvre les tests d'intégration existants, la compilation, des lectures HTTP concurrentes, l'ouverture de connexions WebSocket et un téléchargement PDF de bout en bout.

Les essais ont été exécutés le 26 octobre 2026 sur l'environnement local de développement. L'API répondait sur le port 3000 et PostgreSQL 16.15 sur le port 5433. La machine de test disposait de 16 processeurs logiques et d'environ 15 Gio de mémoire visible. Node.js 22.22.0 et npm 10.9.4 étaient installés. Aucune campagne AWS n'a été exécutée.

## 1.1 Précautions de campagne

Les scénarios de charge HTTP interrogent des routes de lecture. Le test PDF a consommé une seule utilisation d'un lien créé pour l'essai. Le test de 300 connexions WebSocket n'a envoyé aucune édition massive. Ces précautions évitent de créer une file de milliers de travaux PDF dans l'environnement de développement.

# 2 Méthode et critères

Les tests d'intégration utilisent la suite Node.js du dépôt et une base PostgreSQL locale. Les mesures HTTP utilisent 40 requêtes simultanées pour chaque scénario, avec 1 000 requêtes au total. Les latences sont calculées côté générateur, du début de la requête à la lecture complète de la réponse. Les percentiles sont les valeurs observées aux rangs p50, p95 et p99.

Le test WebSocket ouvre 300 connexions vers le même document et attend le message initial de synchronisation sur chaque connexion. Le parcours PDF crée un compte de test, un document, une sauvegarde, un lien limité, réserve une utilisation, attend l'état prêt et télécharge le fichier. La validité minimale du PDF est contrôlée par sa signature binaire « `%PDF-` ».

# 3 Résultats

## 3.1 Compilation et tests d'intégration

La commande `npm run build` a compilé l'API TypeScript et l'interface Vite sans erreur. La suite `npm test` a migré le schéma puis exécuté trois tests :

— fusion d'éditions Yjs simultanées, persistance et diffusion de présence ;
— respect exact du quota de lien sous 40 réservations concurrentes, idempotence, épuisement et calcul du crédit partagé ;
— contrôle des sessions, des rôles et des opérations d'administration.

| Résultat | Nombre | Statut |
|---|---|---|
| Tests réussis | 3 | Réussi |
| Tests échoués | 0 | Réussi |
| Tests ignorés | 0 | Réussi |

La sortie TAP complète est conservée dans `preuves/integration.tap.log`.

## 3.2 Lectures HTTP concurrentes

Les deux routes ont répondu avec le code HTTP 200 pour les 1 000 requêtes de chaque scénario.

| Scénario | Requêtes | Concurrence | Débit (req/s) | p95 (ms) | p99 (ms) |
|---|---|---|---|---|---|
| État de santé `/health` | 1 000 | 40 | 1 533.2 | 67.1 | 103.1 |
| Lecture publique d'un lien | 1 000 | 40 | 2 275.2 | 25.0 | 26.8 |

Pour `/health`, le temps total était de 652.3 ms et la latence p50 de 20.9 ms. Pour la lecture publique, le temps total était de 439.5 ms et la latence p50 de 16.8 ms. Toutes les réponses étaient HTTP 200. Les résultats bruts et le script de reproduction sont fournis dans `preuves/charge-locale.json` et `outils/charge-locale.mjs`. Ces résultats décrivent des lectures sur une instance locale ; ils ne prédisent pas le débit AWS ni celui d'opérations d'écriture.

## 3.3 Connexions WebSocket

Sur 300 connexions ouvertes simultanément vers un document, 300 messages de synchronisation initiaux ont été reçus et aucune connexion n'a échoué. L'ouverture complète a pris 827.8 ms, soit 362.4 connexions par seconde. Les latences d'ouverture mesurées étaient p50 =

311.0 ms, p95 = 742.7 ms et p99 = 791.6 ms. La sortie JSON complète est conservée dans `preuves/charge-locale.json`.

Les 300 sessions utilisaient le même compte de test. Cet essai valide une charge de connexions sur un processus local, mais ne valide pas 300 identités, l'activité continue de 300 éditeurs, la convergence sous édition soutenue ni une architecture multi-instance.

## 3.4 Réservation et fichier PDF

Une réservation a reçu HTTP 202, a atteint l'état `READY` à la révision 1, puis le téléchargement a répondu HTTP 200. Le fichier téléchargé faisait 1 576 octets et commençait par la signature attendue `%PDF-`. Le parcours a pris 511.6 ms entre la première interrogation du statut et le téléchargement. Une seule unité de quota a été consommée. La preuve structurée est conservée dans `preuves/pdf-e2e.json`.

# 4 Analyse et limites

Les résultats confirment les propriétés couvertes par les scénarios exécutés : les invariants de quota et de crédit sous les concurrences ciblées par les tests d'intégration, les contrôles d'accès, le démarrage WebSocket simultané et la génération d'un PDF téléchargeable.

Ils ne démontrent pas les objectifs de charge du cahier des charges. En particulier, aucun essai n'a envoyé 15 000 clics de visiteurs, dont 12 000 dans la première minute, tout en maintenant 300 rédacteurs actifs. Le test WebSocket ne simule ni des identités distinctes ni des modifications répétées. L'environnement utilisé ne comprend pas de campagne de déploiement AWS instrumentée ; S3, SQS, SES, l'équilibrage de charge, la montée en charge des API et des workers ne sont pas validés ici.

Une campagne complète doit être conduite sur un environnement isolé et dimensionné, avec des liens et documents dédiés. Elle devra enregistrer les résultats bruts, les latences p50/p95/p99, le débit, les erreurs fonctionnelles et réseau, la profondeur de file PDF, les ressources consommées, les journaux AWS et les coûts. Les critères d'acceptation doivent distinguer une réponse fonctionnelle (prêt, en attente ou quota épuisé) d'un refus technique ou d'un délai dépassé.

# 5 Conclusion

La validation locale est positive pour les scénarios fonctionnels exécutés et pour une charge modérée de lectures et de connexions WebSocket. La capacité cible du projet n'est pas encore prouvée. Le rapport devra être complété après une campagne de charge complète sur l'environnement de déploiement, sans remplacer les résultats mesurés par une extrapolation des tests locaux.

# A Reproduction des vérifications

Depuis le dossier `application/`, avec PostgreSQL démarré sur le port configuré dans `.env` :

```
npm run build
npm test
```

Les mesures HTTP et WebSocket consignées ici ont été exécutées avec `node '../Rapport de test/outils/charge-locale.mjs'` contre l'API locale sur `http://127.0.0.1:3000`. Le script crée un compte, un document et un lien de test ; il ne réserve aucun téléchargement. Le parcours PDF dédié, reproductible avec `node '../Rapport de test/outils/pdf-e2e.mjs'`, consomme une unité de quota. Les journaux de preuve et leurs empreintes figurent dans `preuves/` et `preuves/SHA256SUMS`.

# B Pièces justificatives

Les fichiers suivants accompagnent ce rapport afin qu'un lecteur puisse vérifier les affirmations et répéter les essais :

— `preuves/build.log` : sortie de compilation API et interface ;
— `preuves/integration.tap.log` : sortie TAP des tests d'intégration ;
— `preuves/charge-locale.json` : résultats bruts des deux charges HTTP et des 300 connexions WebSocket ;
— `preuves/pdf-e2e.json` : statuts HTTP, état final, révision, taille et signature du PDF ;
— `resultats-bruts.json` : résultats consolidés ; les scripts de reproduction sont dans `outils/`.

Les quatre images de `preuves/captures/` sont des rendus visuels en noir et blanc des sorties conservées, et non des captures directes de l'écran. Les journaux bruts restent les pièces de référence. Le fichier `preuves/SHA256SUMS` donne une empreinte SHA-256 des journaux et des images, afin de détecter une modification après génération. Ces pièces ne contiennent ni identifiants de session ni secrets de configuration.

# C Captures visuelles des résultats

Les figures ci-dessous facilitent la lecture des résultats. Chaque figure est dérivée d'un journal ou d'un résultat JSON identifié dans les légendes ; le fichier source permet de vérifier les valeurs exactes.

**Preuve visuelle — compilation**

Source : preuves/build.log (sortie de npm run build)

```
> @journal/web@0.1.0 build
> tsc --noEmit -p tsconfig.json && vite build

vite v6.4.5 building for production...
transforming...
✓ 796 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                              0.47 kB │ gzip:   0.31 kB
dist/assets/index-9FlNPluZ.css              17.94 kB │ gzip:   4.53 kB
dist/assets/CollaborativeEditor-dkj_jvEW.css  25.72 kB │ gzip:   4.03 kB
dist/assets/index-CMcorQtp.js              250.99 kB │ gzip:  77.67 kB
dist/assets/CollaborativeEditor-D7_q8s8E.js  356.37 kB │ gzip: 109.24 kB
✓ built in 2.54s
```

FIGURE C.1 – Sortie de compilation de l'API et de l'interface.

**Preuve visuelle — tests d'intégration**

Source : preuves/integration.tap.log (résumé extrait de la sortie TAP)

```
# Subtest: Yjs fusionne les éditions simultanées et synchronise le contenu durable
ok 1 - Yjs fusionne les éditions simultanées et synchronise le contenu durable
# Subtest: les réservations concurrentes ne dépassent jamais la limite et sont idempotentes
ok 2 - les réservations concurrentes ne dépassent jamais la limite et sont idempotentes
# Subtest: les sessions protègent les comptes et le tableau de bord administrateur
ok 3 - les sessions protègent les comptes et le tableau de bord administrateur
# tests 3
# pass 3
# fail 0
# skipped 0
# duration_ms 30957.528358
```

FIGURE C.2 – Sortie TAP des trois tests d'intégration réussis.

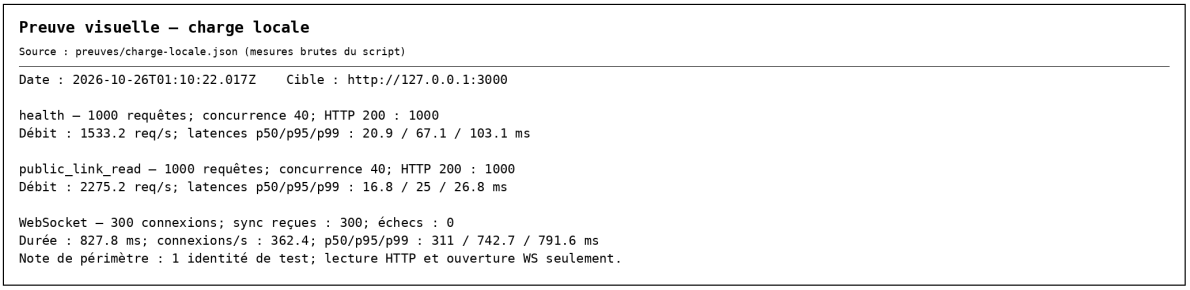



FIGURE C.3 – Résumé visuel des mesures HTTP et des 300 synchronisations WebSocket locales.

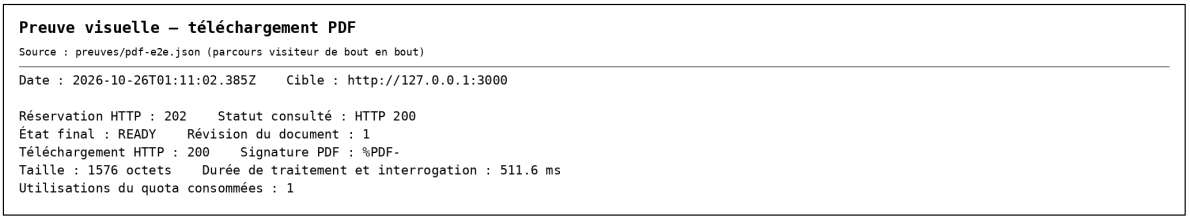



FIGURE C.4 – Réservation visiteur et validation du fichier PDF téléchargé.

# D Justificatif des coûts AWS

Les essais de ce rapport ont été exécutés sur l'environnement local. Aucun déploiement AWS ni accès aux données de facturation AWS n'a été utilisé pendant cette campagne. Il n'est donc pas possible d'y joindre une capture authentique du coût AWS du projet ; ce justificatif demeure à fournir.

Après le déploiement, joindre une capture de Cost Explorer montrant la période, la devise, les filtres et le regroupement par service, puis la page Bills ou la facture correspondante. Cost Explorer est utile pour analyser l'évolution et la répartition des coûts ; la facture clôturée établit le montant final dû. Pour une période en cours, AWS indique que le montant est estimatif et que les données de facturation sont actualisées au moins quotidiennement. Voir la documentation AWS sur les factures et les différences entre les données Billing et Cost Explorer.

Une capture de l'AWS Pricing Calculator peut compléter l'analyse en documentant le coût prévisionnel et ses hypothèses, mais elle ne remplace pas la preuve des frais réellement engagés. Masquer l'identifiant du compte, les coordonnées personnelles et toute information de paiement avant l'insertion.